

LAPORAN
KONSEP CONTAINER DAN INSTALASI DOCKER

Disusun untuk memenuhi tugas
Mata Kuliah : Workshop DevOps
Dosen Pengampu : Ferry Astika Saputra



Oleh :
Mohammad Affan Hendi Firmansyah (3126640049)

KELAS 1
SARJANA TERAPAN LANJUT JENJANG
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

1. TUJUAN PRAKTIKUM

- **Memahami Arsitektur & Isolasi Kontainer:** Mengidentifikasi perbedaan teknis antara *Virtual Machine* (VM) dan kontainer, khususnya peran kernel Linux (*namespaces*, *cgroups* v2, dan *Overlay2*).
- **Instalasi & Konfigurasi Docker Engine:** Mengonfigurasi repositori resmi Docker CE, kunci GPG terverifikasi, instalasi paket inti (docker-ce, containerd.io, buildx, compose), dan hak akses non-root grup docker.
- **Manajemen Siklus Hidup Kontainer:** Mengoperasikan perintah esensial (run, ps, logs, exec, rm) serta memetakan port host-ke-kontainer (*port publishing*).
- **Pembangunan Image Kustom:** Merancang Dockerfile berbasis distribusi minimalis (nginx:1.26-alpine), mengompilasi image pens-web:1.0 via Docker BuildKit, dan memvalidasi akses via port host.
- **Analisis Keamanan DevSecOps:** Mengevaluasi implikasi keamanan keanggotaan grup docker, bahaya tag latest, perbedaan instruksi EXPOSE vs flag -p, serta komparasi *attack surface* Debian vs Alpine.

2. DASAR TEORI: KOMPARASI VIRTUAL MACHINE DAN KONTAINER

Kontainer mengisolasi proses menggunakan fitur kernel sistem operasi host tanpa memerlukan *guest OS* atau *hypervisor*, menjadikannya jauh lebih efisien dibandingkan mesin virtual:

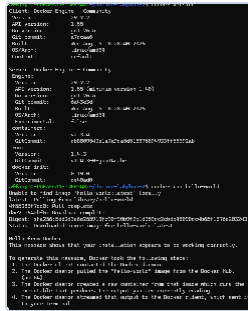
Parameter Evaluasi	Virtual Machine (VM)	Kontainer (Docker)	Analisis Operasional & Keamanan
Arsitektur Kernel	Tiap VM membawa kernel OS sendiri	Berbagi kernel OS host (<i>shared kernel</i>)	Kontainer lebih ringan; VM memiliki isolasi perangkat keras lebih kuat.
Ukuran Image	Gigabytes (GB)	Puluhan hingga ratusan Megabytes (MB)	Kontainer mempercepat transfer jaringan dan efisiensi penyimpanan lokal.
Waktu Startup	Menit (booting full OS)	Milidetik hingga detik (eksekusi proses)	Kontainer memungkinkan <i>auto-scaling</i> elastis dan deployment instan.
Mekanisme Isolasi	Hypervisor (Type 1 / Type 2)	Linux Namespaces, cgroups, AppArmor/Seccomp	Isolasi kontainer tingkat proses; wajib konfigurasi <i>hardening</i> runtime.
Kasus Penggunaan	Multi-OS, isolasi ketat, legacy apps	Microservices, CI/CD, cloud-native apps	Praktik modern sering menjalankan kontainer di dalam VM (<i>hybrid</i>).

3. LANGKAH PRAKTIKUM & BUKTI EKSEKUSI

a. Instalasi Docker Engine & Verifikasi Izin Akses Non-Root

Repositori APT resmi Docker dikonfigurasi menggunakan kunci GPG terverifikasi pada `/etc/apt/keyrings/docker.gpg`. Paket `docker-ce`, `docker-ce-cli`, `containerd.io`, `docker-buildx-plugin`, dan `docker-compose-plugin` dipasang. Akun pengguna lokal dimasukkan ke dalam grup `docker` untuk memungkinkan eksekusi kontainer tanpa `sudo`.

```
sudo usermod -aG docker $USER && newgrp docker
docker version
docker run hello-world
```

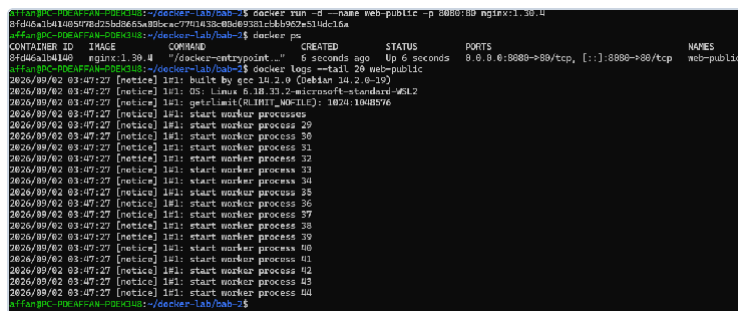


Gambar 1: Output docker version membuktikan integrasi Client dan Server Engine (v29.7.2) serta containerd (v2.3.4), dilanjutkan eksekusi sukses container hello-world.

b. Deployment Nginx, Status Layanan, dan Cuplikan Log

Kontainer Nginx diunduh dan dijalankan di latar belakang (*detached mode*) dengan pemetaan port 8080:80. Status operasional kontainer dan log proses pekerja diverifikasi sebagai bukti sistem bekerja:

```
docker pull nginx:1.30.4
docker run -d --name web-public -p 8080:80 nginx:1.30.4
docker ps && docker logs --tail 20 web-public
```

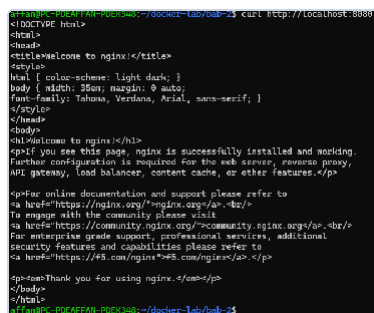


Gambar 2: Output docker ps menunjukkan kontainer web-public aktif (Up 6 seconds, port 8080->80/tcp) dan docker logs mengonfirmasi worker processes siap menerima request.

c. Verifikasi Respon HTTP Server via cURL

Layanan web Nginx yang dipublikasikan pada port 8080 host diuji keterjangkauannya menggunakan utilitas cURL:

```
curl http://localhost:8080
```



Gambar 3: Respons cURL menampilkan payload HTML selamat datang Nginx, mengonfirmasi aturan port forwarding host ke kontainer via iptables aktif.

d. Pembangunan Image Kustom Melalui Dockerfile

Disusun berkas Dockerfile dan index.html kustom pada direktori ~/docker-lab/custom-web berbasis base image minimalis nginx:1.26-alpine:

```
FROM nginx:1.26-alpine
LABEL maintainer="admin@pens.ac.id"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Image dikompilasi menggunakan Docker BuildKit: docker build -t pens-web:1.0 ..

1. Komparasi Base Image: Debian vs Alpine Linux:

- `nginx:1.30.4` (Debian): Ukuran ~140–190 MB, berbasis `glibc`, menyertakan paket utilitas lengkap (`Bash`, `Perl`, `Coreutils`). Memiliki *attack surface* lebih besar dengan potensi kerentanan CVE lebih tinggi.
- `nginx:1.26-alpine` (Alpine): Ukuran ~40–45 MB (hemat 75%), berbasis `musl-libc` dan `BusyBox`. Sangat minim dependensi dan biner sistem, secara drastis memperkecil *attack surface* serta mempersulit penyerang saat tahap *post-exploitation*.

6. REKOMENDASI PERBAIKAN UNTUK LINGKUNGAN PRODUKSI (WAJIB)

1. **Penerapan Pengguna Non-Root:** Tambahkan instruksi pembuatan pengguna non-root pada Dockerfile (`USER appuser`, `UID 10001`) agar proses Nginx tidak berjalan dengan privilege root di dalam namespace kontainer.
2. **Read-Only Root Filesystem:** Jalankan kontainer dengan flag `--read-only` untuk mencegah modifikasi biner oleh penyerang, dan alokasikan `--tmpfs` pada direktori temporer seperti `/var/run` dan `/var/cache/nginx`.
3. **Pembatasan Sumber Daya (cgroups v2):** Terapkan batas memori dan CPU eksplisit (`--memory="256m" --cpus="0.5" --pids-limit 100`) guna mencegah serangan *Denial of Service* (DoS) dan fenomena kehabisan memori host (*OOM killer*).
4. **Vulnerability Scanning & Image Signing:** Integrasikan pemindai kerentanan seperti **Trivy** pada pipeline CI/CD untuk memblokir image dengan temuan kerentanan *HIGH* atau *CRITICAL*, serta tandatangani image menggunakan **Cosign** (**Sigstore**) untuk menjamin integritas asal artefak.

7. EVALUASI DAN LATIHAN MANDIRI

1. Mengapa penggunaan tag `latest` tidak dianjurkan untuk deployment yang harus *reproducible*?

Jawaban: Tag `latest` adalah alias dinamis (*mutable pointer*) yang selalu menunjuk pada build terakhir. Dua build atau deployment yang dieksekusi pada waktu berbeda dengan kode sumber sama dapat menghasilkan artefak yang berbeda jika base image telah diperbarui oleh vendor, sehingga merusak replikasi lingkungan yang konsisten dan menyulitkan proses rollback.

1. Jelaskan peran `containerd` dan `runc` dalam arsitektur Docker.

Jawaban: `containerd` adalah *high-level container runtime* yang mengelola seluruh siklus hidup kontainer (`start/stop/pause`), transfer image dari registry, serta manajemen storage dan network. Sedangkan `runc` adalah *low-level OCI runtime* yang berinteraksi langsung dengan kernel Linux untuk membuat namespace, menetapkan limit `cgroups`, mengonfigurasi filter `seccomp`, dan mengeksekusi proses kontainer sebelum keluar (*exit*).

1. Apa konsekuensi keamanan dari memasukkan user ke group `docker`?

Jawaban: Memasukkan akun ke grup `docker` memberikan akses tulis ke socket `/var/run/docker.sock`. Karena Docker daemon berjalan dengan hak root, hal ini ekuivalen dengan memberikan hak root tanpa kata sandi (*unrestricted root equivalent*). Pengguna dapat dengan mudah mengambil alih host dengan `me-mount` direktori root host ke dalam kontainer.

1. Bandingkan layer image `nginx:1.26-alpine` dan image custom yang Anda buat.

Jawaban: Image `nginx:1.26-alpine` tersusun dari layer `rootfs` minimalis Alpine Linux, instalasi biner Nginx, konfigurasi default, dan direktori web standar. Image kustom `pens-web:1.0` menggunakan base image tersebut sebagai pondasi, kemudian menambahkan satu layer baru di atasnya melalui instruksi `COPY index.html ...` yang berisi artefak halaman statis kustom, serta menyematkan metadata `LABEL` dan `EXPOSE 80`.

1. Kapan sebaiknya memilih VM daripada container?

Jawaban: VM sebaiknya dipilih ketika: (1) Memerlukan isolasi keamanan *multi-tenant* tingkat perangkat keras (*hardware-level boundary*); (2) Menjalankan sistem operasi dengan kernel berbeda dari host (misal menjalankan Windows di host Linux); (3) Memerlukan akses modul kernel khusus atau driver perangkat keras tingkat rendah; atau (4) Menjalankan aplikasi *monolithic legacy* yang tidak dirancang untuk lingkungan modular.

8. KESIMPULAN

Praktikum Bab 2 berhasil membuktikan prinsip fundamental kontainerisasi, instalasi komponen Docker CE (v29.7.2), serta pengelolaan siklus hidup kontainer pada Ubuntu 26.04 LTS (WSL2). Kontainer terbukti jauh lebih ringan dan efisien dibandingkan VM karena memanfaatkan *shared kernel* melalui abstraksi *namespaces* dan *cgroups*. Pembangunan image kustom `pens-web:1.0` berbasis Alpine Linux mendemonstrasikan efisiensi arsitektur *layering* UnionFS dengan ukuran ringkas (~45 MB) dan build time singkat (4.8 detik). Dari aspek DevSecOps, praktikum menegaskan pentingnya pembatasan hak grup `docker`, penghindaran tag `latest`, pemahaman peran metadata `EXPOSE` vs `runtime -p`, serta pemilihan base image minimalis guna memperkecil *attack surface* di lingkungan produksi.